S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 11
Postlab (OPEN ENDED PRACTICAL)

Aim: Implementing a Virtual Memory System with Demand Paging where pages are loaded into memory only when needed, optimizing memory usage and minimizing page faults.

Name: Aaditya S Gudhniye

USN: CM24062

Semester / Year:

Academic Session:

Date of Performance:

Date of Submission:

- **Aim:** Implementing a Virtual Memory System with Demand Paging where pages are loaded into memory only when needed, optimizing memory usage and minimizing page faults.

- **Objectives:**

i. Implement demand paging to load pages only when required.
ii. Optimize memory utilization and reduce page faults.
iii. Simulate page replacement algorithms for efficient management.

- **Requirements:**
  - ✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

  - ✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

- **Theory:**

What is Demand Paging?

Demand paging is a technique used in virtual memory systems where pages enter main memory only when requested or needed by the CPU. In demand paging, the operating system loads only the necessary pages of a program into memory at runtime, instead of loading the entire program into memory at the start. A page fault occurred when the program needed to access a page that is not currently in memory.

The operating system then loads the required pages from the disk into memory and updates the page tables accordingly. This process is transparent to the running program and it continues to run as if the page had always been in memory.

Pure Demand Paging

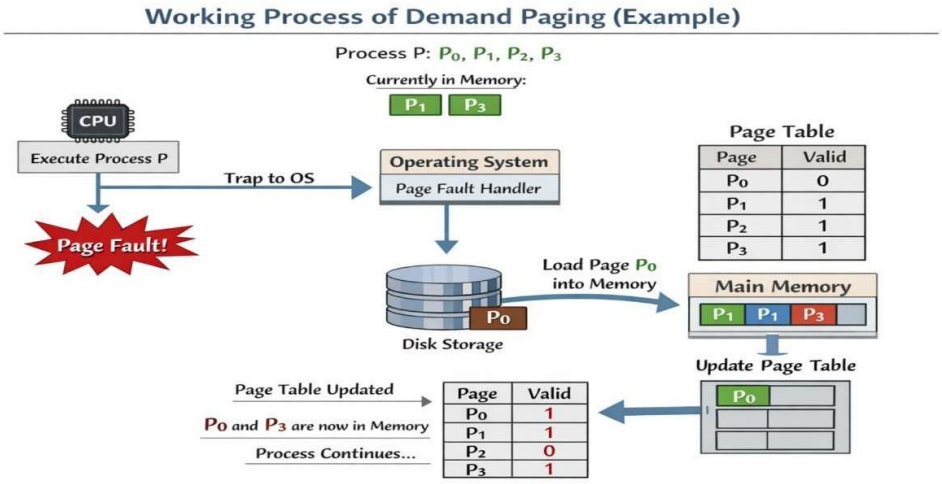
Pure demand paging is a specific implementation of demand paging. The operating system only loads pages into memory when the program needs them. In on-demand paging only, no pages are initially loaded into memory when the program starts, and all pages are initially marked as being on disk.

Operating systems that use pure demand paging as a memory management strategy do so without preloading any pages into physical memory prior to the commencement of a task. Demand paging loads a process's whole address space

into memory one step at a time, bringing just the parts of the process that are actively being used into memory from disc as needed.
It is useful for executing huge programs that might not fit totally in memory or for computers with limited physical memory. If the program accesses a lot of pages that are not in memory right now, it could also result in a rise in page faults and possible performance overhead. Operating systems frequently use caching techniques and improve page replacement algorithms to lessen the negative effects of page faults on system performance as a whole.

Working Process of Demand Paging

Let us understand this with the help of an example. Suppose we want to run a process P which have four pages P0, P1, P2, and P3. Currently, in the page table, we have pages P1 and P3.



The operating system's demand paging mechanism follows a few steps in its operation.

- **Program Execution:** Upon launching a program, the operating system allocates a certain amount of memory to the program and establishes a process for it.
- **Creating Page Tables:** To keep track of which program pages are currently in memory and which are on disk, the operating system makes page tables for each process.
- **Handling Page Fault:** When a program tries to access a page that isn't in memory at the moment, a page fault happens. In order to determine whether the necessary page is on disk, the operating system pauses the application and consults the page tables.
- **Page Fetch:** The operating system loads the necessary page into memory by retrieving it from the disk if it is there.
- The page's new location in memory is then reflected in the page table.

- **Resuming The Program:** The operating system picks up where it left off when the necessary pages are loaded into memory.
- **Page Replacement:** If there is not enough free memory to hold all the pages a program needs, the operating system may need to replace one or more pages currently in memory with pages currently in memory. on the disk. The page replacement algorithm used by the operating system determines which pages are selected for replacement.
- **Page Cleanup:** When a process terminates, the operating system frees the memory allocated to the process and cleans up the corresponding entries in the page tables.

Advantages of Demand Paging

So in the Demand Paging technique, there are some benefits that provide efficiency of the operating system.

- **Efficient use of physical memory**: Query paging allows for more efficient use because only the necessary pages are loaded into memory at any given time.
- **Support for larger programs:** Programs can be larger than the physical memory available on the system because only the necessary pages will be loaded into memory.
- **Faster program start:** Because only part of a program is initially loaded into memory, programs can start faster than if the entire program were loaded at once.
- **Reduce memory usage:** Query paging can help reduce the amount of memory a program needs, which can improve system performance by reducing the amount of disk I/O required.

Disadvantages of Demand Paging

- **Page Fault Overload:** The process of swapping pages between memory and disk can cause a performance overhead, especially if the program frequently accesses pages that are not currently in memory.
- **Degraded Performance:** If a program frequently accesses pages that are not currently in memory, the system spends a lot of time swapping out pages, which degrades performance.
- **Fragmentation:** Query paging can cause physical memory fragmentation, degrading system performance over time.
- **Complexity:** Implementing query paging in an operating system can be complex, requiring complex algorithms and data structures to manage page tables and swap space.

What is the Impact of Demand Paging in Virtual Memory Management?

With demand paging, the operating system swaps memory pages between the main memory and secondary storage based on need. When a program needs a page not currently in memory, the operating system retrieves it from secondary storage, a process called a page fault.

Demand paging significantly impacts virtual memory management by allowing the operating system to use virtual memory efficiently, improving overall system performance. Its main advantage is reducing the physical memory required, enabling more applications to run at once and allowing larger programs to run.
However, demand paging has some drawbacks. The page fault mechanism can delay program execution because the operating system must retrieve pages from secondary storage. This delay can be minimized by optimizing the page replacement algorithm.

Virtual Memory vs Physical Memory

Feature	**Virtual Memory**	**Physical Memory (RAM)**
Definition	An abstraction that extends the available memory by using disk storage	The actual hardware (RAM) that stores data and instructions currently being used by the CPU
Location	On the hard drive or SSD	On the computer's motherboard
Speed	Slower (due to disk I/O operations)	Faster (accessed directly by the CPU)
Capacity	Larger, limited by disk space	Smaller, limited by the amount of RAM installed
Cost	Lower (cost of additional disk storage)	Higher (cost of RAM modules)
Data Access	Indirect (via paging and swapping)	Direct (CPU can access data directly)
Volatility	Non-volatile (data persists on disk)	Volatile (data is lost when power is off)

❖ **CODE:**

```
#include <stdio.h>

#define MAX_PAGES 10
#define MAX_FRAMES 3

// Page table structure
typedef struct {
    int frame;
    int valid;  // 0 = not in memory, 1 = in memory
} PageTable;

PageTable pageTable[MAX_PAGES];
int frames[MAX_FRAMES];
int frameIndex = 0;
int pageFaults = 0;

// Function to initialize page table
void initialize() {
    for (int i = 0; i < MAX_PAGES; i++) {
        pageTable[i].valid = 0;
        pageTable[i].frame = -1;
    }

    for (int i = 0; i < MAX_FRAMES; i++) {
        frames[i] = -1;
    }
}

// FIFO Page Replacement
void loadPage(int page) {
    printf("Page Fault! Loading page %d...\n", page);

    // Replace using FIFO
    int replacedPage = frames[frameIndex];

    if (replacedPage != -1) {
        pageTable[replacedPage].valid = 0;
        printf("Replacing page %d\n", replacedPage);
    }

    frames[frameIndex] = page;
```

```
    pageTable[page].frame = frameIndex;
    pageTable[page].valid = 1;
    frameIndex = (frameIndex + 1) % MAX_FRAMES;
    pageFaults++;
}

// Access page
void accessPage(int page) {
    if (pageTable[page].valid == 1) {
        printf("Page %d is already in memory (Frame %d)\n",
               page, pageTable[page].frame);
    } else {
        loadPage(page);
    }
}

// Display memory state
void displayMemory() {
    printf("Current Frames: ");
    for (int i = 0; i < MAX_FRAMES; i++) {
        if (frames[i] != -1)
            printf("%d ", frames[i]);
        else
            printf("- ");
    }
    printf("\n");
}
int main() {
    initialize();
    // Example reference string
    int referenceString[] = {1, 3, 0, 1, 2, 3, 0, 4};
    int n = sizeof(referenceString) / sizeof(referenceString[0]);

    printf("Demand Paging Simulation\n\n");

    for (int i = 0; i < n; i++) {
        printf("Accessing page %d\n", referenceString[i]);
        accessPage(referenceString[i]);
        displayMemory();
        printf("\n");
    }
    printf("Total Page Faults = %d\n", pageFaults);
    return 0;
     }
```

❖ **Output:**

main.c

```c
// Online C compiler to run C program online
#include <stdio.h>

#define MAX_PAGES 10
#define MAX_FRAMES 3

// Page table structure
typedef struct {
    int frame;
    int valid;  // 0 = not in memory, 1 = in memory
} PageTable;

PageTable pageTable[MAX_PAGES];
int frames[MAX_FRAMES];
int frameIndex = 0;
int pageFaults = 0;

// Function to initialize page table
void initialize() {
    for (int i = 0; i < MAX_PAGES; i++) {
        pageTable[i].valid = 0;
        pageTable[i].frame = -1;
    }

    for (int i = 0; i < MAX_FRAMES; i++) {
        frames[i] = -1;
    }
}

// FIFO Page Replacement
```

Output

```
Demand Paging Simulation

Accessing page 1
Page Fault! Loading page 1...
Current Frames: 1 - -

Accessing page 3
Page Fault! Loading page 3...
Current Frames: 1 3 -

Accessing page 0
Page Fault! Loading page 0...
Current Frames: 1 3 0

Accessing page 1
Page 1 is already in memory (Frame 0)
Current Frames: 1 3 0

Accessing page 2
Page Fault! Loading page 2...
Replacing page 1
Current Frames: 2 3 0

Accessing page 3
Page 3 is already in memory (Frame 1)
Current Frames: 2 3 0

Accessing page 0
Page 0 is already in memory (Frame 2)
Current Frames: 2 3 0
```

❖ **Conclusion**: In this practical, we conclude that demand paging efficiently manages memory by loading pages only when needed, reducing page faults and optimizing system performance.

❖ **Discussion Questions:**

1. **What is Demand Paging in Virtual Memory?**
2. **What causes a page fault in Demand Paging?**
3. **What are the advantages of Demand Paging?**
4. **Which page replacement algorithms are commonly used in Demand Paging?**
5. **How does Demand Paging differ from Pre-paging?**

❖ **References:**

https://www.geeksforgeeks.org/disk-scheduling-algorithms/
https://chatgpt.com/

Date:____/____/2026

Signature
Course Coordinator
B.Tech CSE(AIML)
Sem: 4 / 2025-26